

Machine Learning Project Report 2025

Intrusion detection systems (IDS) using Machine Learning



Théo HARDY
Gaël LE MOUEL
Adam FERRE

December 12, 2025

GitHub repository

Contents

1	Business Scope and Problem Formalization	3
1.1	Business Case	3
1.2	Problem Formalization	3
2	Data Description and Exploration	3
2.1	Dataset Source	3
2.2	Exploratory Data Analysis (EDA)	4
3	Methodology	5
3.1	Preprocessing	5
3.2	Algorithm Description	5
3.2.1	Logistic Description	5
3.2.2	Random Forest	5
3.2.3	Logistic Regression with PCA	5
3.2.4	Grid Search	5
3.2.5	WDRO	6
3.2.6	Projected Gradient Descent (PGD) Adversarial Training	6
4	Algorithm Implementation and Results	6
4.1	Logistic Regression with PCA Results	6
4.1.1	Grid Search	8
4.1.2	Cross validation	8
4.1.3	WDRO	9
5	Discussion and Limitations	10
5.1	Addressing Obstacles	10
5.2	Limitations	10
6	Conclusion	11

1 Business Scope and Problem Formalization

1.1 Business Case

The rapid proliferation of Internet of Things (125 billion interconnected IoT devices in 2030 [1]) in sectors such as healthcare, smart cities, and industry has created a vast new attack surface for cybercriminals. As highlighted by recent research, these devices are frequently targeted by varied and sophisticated attacks, such as Distributed Denial of Service (DDoS)[2] and SSH Brute-Force exploits [3], often due to weak default security configurations.

While detecting an anomaly is a critical first step, a robust security system must also identify the specific type of attack to automate the correct response. For instance, a DDoS attack requires traffic rate-limiting, whereas an SSH Brute-Force attack requires blocking specific IP addresses or locking credentials.

Consequently, the business objective of this project is to build a robust, multi-class Intrusion Detection System (IDS). Unlike the original research which focused on lightweight deployment for resource-constrained devices, our project pivots to prioritize model robustness and granular identification. The goal is not merely to detect intruders, but to accurately categorize network traffic into specific normal profiles and distinct attack classes, ensuring precise and targeted mitigation strategies.

1.2 Problem Formalization

We formalize this project as a Multiclass Classification problem. Unlike simple binary detection, our model must distinguish between benign behaviors and specific attack signatures to ensure the security of the IoT infrastructure.

- **Input (X):** Network traffic features derived from the RT-IoT2022 dataset[4], such as packet inter-arrival times, flow duration, and flag counts.
- **Output (Y):** A categorical label indicating one of several classes:
 - **Normal Traffic :** Traffic from specific devices including ThingSpeak-LED, MQTT-Temp, Wipro-Bulb, and Amazon-Alexa.
 - **Attack Traffic :** Including DDoS, SSH Brute-Force, SYN Flood, UDP Flood, and Scan attacks.

Our objective is to maximize the overall Macro-Averaged F1-Score. This metric is crucial because it ensures the model performs well across all classes, preventing the "robustness" score from being skewed by the most frequent traffic type (class imbalance).

2 Data Description and Exploration

2.1 Dataset Source

The dataset used in this project is the RT-IoT2022 (Real-Time Internet of Things 2022) dataset, developed by Sharmila and Nagapadma (2023) [4]. Unlike many synthetic datasets, RT-IoT2022 was generated using a physical testbed infrastructure comprising

real-time IoT devices, ensuring the network traffic reflects authentic behavior rather than simulated patterns.

The data collection infrastructure consisted of two distinct networks connected via a router: a victim network of IoT devices and an attacker network. The specific IoT devices used to generate benign (normal) traffic include:

- **ThingSpeak-LED** - An IoT LED light controlled via the ThingSpeak platform.
- **MQTT - Temp** - A Raspberry Pi device transmitting temperature sensor data to a Mosquitto Broker via the MQTT protocol.
- **Wipro-Bulb** A smart home integrated solution (NS9400 9-Watt B22) controlled via WiFi.
- **Amazon-Alexa** -An Echo Dot device communicating with cloud-based voice services.

Raw network traffic was captured using Wireshark¹ and processed using the CICFlowmeter² tool. This process converted raw packet data into bidirectional flow-based features (e.g., Flow Duration, Packet Inter-arrival Time, Flag Counts). To prevent model bias, specific identifiers such as source/destination IP addresses and Flow IDs were removed during preprocessing.

2.2 Exploratory Data Analysis (EDA)

To ensure the data was suitable for machine learning applications, we examined several key components, starting with the feature structure. It was observed that the dataset features are not raw packet data but rather pre-computed statistical aggregations (including mean, standard deviation, and max/min values) generated by the extraction tool. Consequently, our analysis focused on understanding how these aggregated metrics—such as `flow_duration`, `idle.std`, or `packet_inter_arrival_time` between normal and anomalous behaviors.

A systematic integrity check was conducted, confirming that the dataset is pristine and fully populated, with no missing values or null entries. As a result, no imputation or data cleaning procedures were required, allowing us to proceed directly to distribution analysis.

The most critical finding during this phase was the significant Class Imbalance within the target labels. Our distribution analysis revealed a stark disparity all of the target labels. This imbalance is a crucial characteristic that directly influenced our modeling strategy, necessitating the use of weighted evaluation metrics (such as the Macro-Averaged F1-Score) and stratified splitting techniques to prevent model bias.

Finally, correlations between features were examined to identify redundant variables or multicollinearity common in flow-based datasets, while potential outliers were reviewed to distinguish between measurement errors and genuine anomalous network behaviors relevant to cybersecurity analysis.

¹<https://www.wireshark.org/>

²<https://github.com/ahlashkari/CICFlowMeter>

3 Methodology

3.1 Preprocessing

As said in 2.2, the dataset was already preprocessed. It was cleaned and required no missing value imputation or duplicate removal. However, feature scaling and label encoding were performed using `StandardScaler`, `LabelEncoder` and `OneHotEncoder` from the sklearn library. This ensured that all features were on a comparable scale, which is particularly important for distance-based algorithms.

3.2 Algorithm Description

For the algorithm implementation, we selected a combination of standard models presented during our Machine Learning course at ESILV and advanced machine learning models to address the multiclass classification problem effectively.

3.2.1 Logistic Description

First, we implemented Logistic Regression, a widely used statistical model for binary and multiclass classification tasks. Logistic Regression estimates the probability of a data point belonging to a particular class using the logistic function. It is particularly effective for linearly separable data and provides interpretable coefficients that indicate the influence of each feature on the predicted outcome.

3.2.2 Random Forest

We also employed Random Forest, an ensemble learning method that constructs multiple decision trees during training and outputs the mode of the classes for classification tasks. Random Forests are known for their robustness to overfitting, ability to handle high-dimensional data, and effectiveness in capturing complex interactions between features.

3.2.3 Logistic Regression with PCA

As our dataset contains a large number of features, we applied Principal Component Analysis (PCA) as a dimensionality reduction technique before training the Logistic Regression model. PCA transforms the original features into a new set of 10 uncorrelated variables (principal components) that capture the maximum variance in the data. This helps to reduce noise and improve model performance by focusing on the most informative aspects of the dataset.

3.2.4 Grid Search

To optimize the hyperparameters of our models, we utilized Grid Search, a systematic approach to exhaustively search through a specified subset of hyperparameters. By evaluating model performance across different combinations of hyperparameters using cross-validation, Grid Search helps identify the optimal settings that yield the best performance on the validation set. This method is particularly useful for tuning models like Logistic Regression and Random Forest, where parameters such as regularization strength and the number of trees can significantly impact results.

3.2.5 WDRO

Wasserstein Distributionally Robust Optimization (WDRO) is a mathematical framework that can provide robustness to data shifts in machine learning models. Distributionally Robust Optimization (DRO) introduces a solution by considering the worst expectation of the cost function when the probability distribution belongs to a neighborhood around the empirical distribution.

A recent push has been made towards wider neighborhoods notions for DRO led by the WDRO formulation. Its ambiguity set relies on the Wasserstein optimal transport cost. We consider an ambiguity set a "ball" of plausible distributions centered around a nominal distribution (e.g., the empirical distribution of training data), where the radius of the ball is defined by the Wasserstein distance. WDRO seeks to minimize the worst-case risk over all distributions within this Wasserstein ball, ensuring that the model performs well even if the true distribution differs from the training data within the specified radius. [5], [6], [7]

3.2.6 Projected Gradient Descent (PGD) Adversarial Training

To get a robust model we looked at adversarial attacks and particularly the Projected Gradient Descent attack proposed Madry et al [8].

The Projected Gradient Descent attack is an iterative method in which, after each iteration, the perturbation is projected on an l_p -ball of specified radius (in addition to clipping the values of the adversarial sample so that it lies in the permitted data range).

Unfortunately, due to errors in our notebook we did not manage to correctly implement this attack and thus evaluate our model through adversarial attacks.

4 Algorithm Implementation and Results

4.1 Logistic Regression with PCA Results

We have pretty good results with a 97 % accuracy and a f1-score of 0.976. The only weakness we find is the macro average f1-score: 72%. It tells us that our model has a pretty good accuracy except for a few classes. We have displayed the confusion matrix (Figure 1) to better visualize the results.

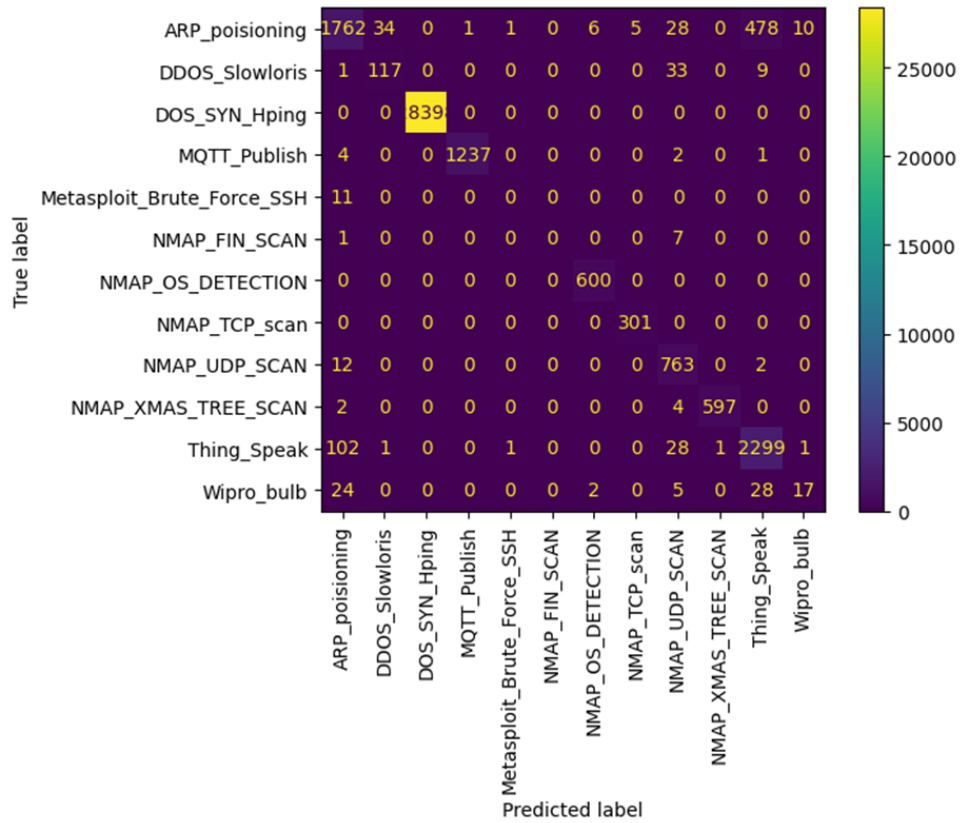


Figure 1: Confusion Matrix of Logistic Regression with PCA

Our model performs well in most attack categories. It is particularly effective at detecting:

- Denial-of-Service (DoS) attacks generated with the Hping tool,
- High-volume MQTT PUBLISH floods,
- OS fingerprinting using Nmap, enabling tailored attacker techniques,
- TCP/UDP port scans used to discover open services

However, it has some weaknesses in detecting:

- Attacks on Wipro lightbulb to expose wi-fi network (33% detection),
- Open port using FIN flag with Nmap (0% detection),
- Brute force attacks on SSH using Metasploit (0% detection).

With the confusion matrix we can also say that our rate of false positive is greater than the rate of false negative. Indeed, in our problem we better have false alarms (false positive) than miss an attack (false negative). To analyse further our model, we are going to analyse the Receiver Operating Characteristic (ROC) curve and Area Under Curve (AUC) score.

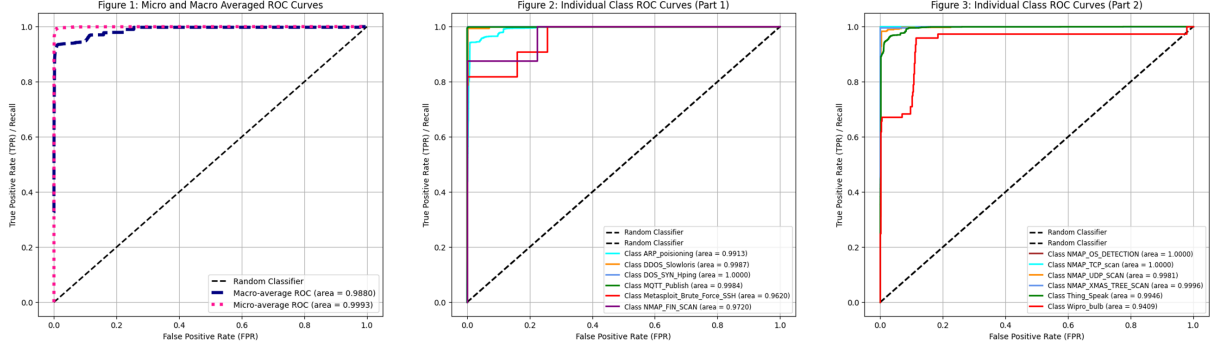


Figure 2: ROC Curve of Logistic Regression with PCA

With the first figure we can clearly see the drawback for having an imbalanced data: we have a nearly perfect micro average curve but the macro average curve displays more false negative due to the imbalanced dataset.

In the second and third figure we can clearly see the 3 attacks that our model has some difficulty to classify: **Metasploit_Brute_Force_SSH**, **NMAP_FIN_SCAN** and **Wipro_bulb**.

4.1.1 Grid Search

To optimize model performance, we initially considered an exhaustive **GridSearchCV** however, given the significant computational cost associated with searching high-dimensional parameter spaces on a large dataset, we adopted a more efficient optimization strategy.

First, we transitioned to **RandomizedSearchCV**, a hyperparameter tuning technique that samples a fixed number of parameter settings (**n_iter**) from specified distributions. Unlike standard grid search, which evaluates every possible combination, randomized search allows for a broad exploration of the parameter space with significantly reduced computational overhead. To further limit training time, we narrowed the hyperparameter search space and reduced the cross-validation strategy to 3-fold validation.

We evaluated four distinct algorithms: **DecisionTreeClassifier**, **RandomForestClassifier**, **SVC**, and **BaggingClassifier**.

All optimized models demonstrated robust performance, achieving accuracies exceeding 96%. Among these candidates, the **BaggingClassifier** proved to be the superior model, yielding a near-perfect accuracy of 99.8% and an F1-score of 0.998. This suggests that the ensemble approach of bagging successfully mitigated variance while retaining high predictive power. While the **BaggingClassifier** achieved an exceptional accuracy of 99.8%, such near-perfect results warrant critical scrutiny regarding potential overfitting. Although Bagging is designed to reduce variance, a score this high may indicate that the model has memorized specific traffic patterns unique to the limited number of devices in our testbed, rather than learning generalizable attack signatures. In a real-world deployment with diverse IoT hardware, this performance would likely degrade.

4.1.2 Cross validation

To optimize the Logistic Regression model with PCA before, we performed Cross Validation.

Metrics	Base model	Mean of Cross validation models
Accuracy	0.977	0.973
Precision	0.977	0.973
Recall	0.977	0.973
F1-score	0.976	0.971

Table 1: Comparison of baseline model and the mean model after cross validation

As we can see after cross validation we lose about 0.4% to 0.5% in all metrics. Using cross validation help us to avoid overfitting and have a more robust model.

The performance of all models was high, with ensemble methods (Random Forest, Gradient Boosting, and Extra Trees) achieving the best results.

The data set is not really balanced so we tried several different models with an imbalanced data set strategy in mind. And we could see some improvement with SMOTE. But as our data were clean, we couldn't really see any major differences.

4.1.3 WDRO

We define 5 Robustness radius (ρ) 2 parameters so that we can train 5 models and compare them.

Robustness radius (ρ)	Accuracy	Weighted Precision	Weighted Recall	Weighted F1-score
10^{-5}	0.966	0.966	0.966	0.965
10^{-4}	0.969	0.968	0.969	0.967
0.001	0.966	0.966	0.966	0.965
0.01	0.933	0.946	0.933	0.926
0.1	0.908	0.885	0.908	0.890

Table 2: Performance metrics for different WDRO robustness radius values

We want to evaluate the robustness of our models so we added Gaussian noise to our test set and see how the models behave.

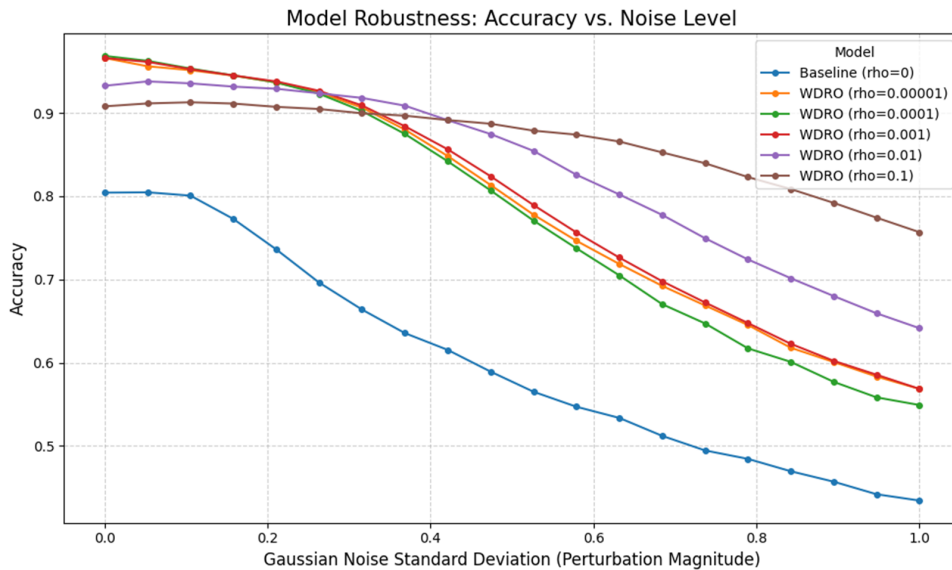


Figure 3: Robustness evaluation of WDRO models with Gaussian noise

As we can see in figure 3 the model with the least robustness radius ($\rho = 10^{-5}$) is the one that performs the best with added noise. As the robustness radius increases the model performance decrease with added noise. It is probably due to the fact that with a high robustness radius the model is too much generalized and thus doesn't perform well on the clean test set nor on the noisy test set.

5 Discussion and Limitations

5.1 Addressing Obstacles

Throughout the project, we encountered several challenges inherent to cybersecurity datasets and machine learning deployment.

Class Imbalance: As noted in the EDA, the dataset exhibited a significant disparity between normal traffic and specific attack classes (e.g., SSH Brute Force). To address this, we shifted our evaluation metric from simple accuracy to the Macro-Averaged F1-Score, ensuring that minority classes were not ignored. We also experimented with SMOTE (Synthetic Minority Over-sampling Technique) to artificially balance the training set. While it provided minor improvements, the high quality of the cleaned dataset meant that standard ensemble methods like Random Forest handled the imbalance relatively well without aggressive resampling.

High Dimensionality: The raw dataset contained numerous flow-based features, which increased computational cost and noise. We successfully addressed this by implementing Principal Component Analysis (PCA), reducing the feature space to 10 uncorrelated components. This not only accelerated training times for the Logistic Regression model but also helped in visualizing the data structure.

Model Robustness: A key objective was ensuring the model's reliability under noise. Standard models often fail when data drifts slightly from the training distribution. By implementing Wasserstein Distributionally Robust Optimization (WDRO), we successfully created a model that maintains performance even when subjected to Gaussian noise. Our experiments showed that a small robustness radius ($\rho = 10^{-5}$) offered the best trade-off, protecting against noise without being overly conservative.

5.2 Limitations

Despite the strong performance metrics, our project has several limitations that must be acknowledged:

- **Dataset Scope:** As mentioned, the RT-IoT2022 dataset is limited to only two major attack types (DDoS and SSH Brute-Force) and four specific IoT devices. This restricts the model's ability to generalize to other common threats like Ransomware or SQL Injection, or to different IoT hardware architectures.
- **Adversarial Vulnerability:** We aimed to evaluate our models against active adversarial attacks using Projected Gradient Descent (PGD). However, due to implementation errors in the attack pipeline, we could not quantify our model's resilience against intelligent adversaries who deliberately perturb inputs to evade detection.
- **Potential Overfitting:** The BaggingClassifier achieved a near-perfect accuracy of 99.8%. While impressive, such a score in a controlled testbed often indicates

overfitting. The model likely memorized specific traffic signatures of the four test devices rather than learning generalized attack behaviors, which would likely lead to performance degradation in a diverse, real-world network.

6 Conclusion

The objective of this project was to develop a robust, multi-class Intrusion Detection System (IDS) capable of securing IoT environments. Using the RT-IoT2022 dataset, we implemented and compared several architectures, ranging from interpretable Logistic Regression to complex Ensemble methods and Robust Optimization techniques.

Our results demonstrate that machine learning can effectively classify IoT network traffic with high precision. All implemented models achieved exceptional performance metrics, with F1-scores consistently exceeding 0.96. While the BaggingClassifier yielded the highest raw score (0.998), the uniformly high performance across diverse algorithms suggests a strong signal in the dataset, but also raises valid concerns regarding potential overfitting to the specific devices in our testbed.

Furthermore, our experiments with WDRO highlighted the critical distinction between accuracy and robustness. We found that while standard models perform exceptionally well on clean data, their performance degrades under noise. The WDRO framework provided a mechanism to mitigate this, though it requires careful tuning of the uncertainty radius (ρ) to avoid over-conservatism.

Finally, it is important to recall that the RT-IoT2022 dataset was originally designed to benchmark lightweight models for resource-constrained devices [4]. While our project prioritized algorithmic robustness, the logical next step would be to apply optimization techniques—such as quantization or pruning—to our best-performing models. As demonstrated by the original authors, reducing the model size would allow our robust IDS to be deployed directly on edge hardware (e.g., Raspberry Pi) with minimal latency and power consumption.

Scientific References

- [1] C. Saadouni, S. El Jaouhari, N. Tamani, S. Ziti, L. Mroueh, and K. El Bouchti, “Identification techniques in the internet of things: Survey, taxonomy and research frontier,” *IEEE Communications Surveys & Tutorials*, 2025.
- [2] A. A. Khan, A. A. Laghari, Z. A. Shaikh, Z. Dacko-Pikiewicz, and S. Kot, “Internet of things (iot) security with blockchain technology: A state-of-the-art review,” *IEEE Access*, vol. 10, pp. 122 679–122 695, 2022.
- [3] G. Fahrnberger, “Realtime risk monitoring of ssh brute force attacks,” in *International conference on innovations for community services*, Springer, 2022, pp. 75–95.
- [4] B. Sharmila and R. Nagapadma, “Quantized autoencoder (qae) intrusion detection system for anomaly detection in resource-constrained iot devices using rt-iot2022 dataset,” *Cybersecurity*, vol. 6, no. 1, p. 41, 2023.
- [5] P. Mohajerin Esfahani and D. Kuhn, “Data-driven distributionally robust optimization using the wasserstein metric: Performance guarantees and tractable reformulations,” *Mathematical Programming*, vol. 171, no. 1, pp. 115–166, 2018.
- [6] R. Gao, X. Chen, and A. J. Kleywegt, “Wasserstein distributionally robust optimization and variation regularization,” *Operations Research*, vol. 72, no. 3, pp. 1177–1191, 2024.
- [7] D. Kuhn, P. M. Esfahani, V. A. Nguyen, and S. Shafieezadeh-Abadeh, “Wasserstein distributionally robust optimization: Theory and applications in machine learning,” in *Operations research & management science in the age of analytics*, Informs, 2019, pp. 130–166.
- [8] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards deep learning models resistant to adversarial attacks,” *arXiv preprint arXiv:1706.06083*, 2017.